

Módulo 1

Fundamentos de DevOps y CI/CD

Cultura CALMS · CI/CD · Toolchain

Clase 1 · Duración 2 h · Nivel inicial

```
devops@coderhouse  
  
$ git commit -m "clase 1"  
$ docker build .  
$ kubectl apply -f app.yaml  
$ terraform apply  
  
✓ ready
```

Agenda de hoy

Dos horas, cuatro bloques y un break de 10 minutos

1

Por qué existe DevOps

El muro entre Dev y Ops, y qué costaba mantenerlo

20 min

2

La cultura CALMS

Los cinco pilares + métricas DORA

30 min



Break

Estiren las piernas

10 min

3

CI/CD en profundidad

Integración continua, delivery vs deployment, pipeline

30 min

4

El toolchain

Git, Docker, Kubernetes, Terraform + demo en vivo

30 min

Al terminar la clase vas a poder...

Estos son los criterios con los que se evalúa el módulo

- 1 Explicar qué es DevOps**
y por qué es una cultura antes que un conjunto de herramientas
- 2 Nombrar los 5 pilares de CALMS**
y dar un ejemplo concreto de cada uno
- 3 Distinguir CI, CD (delivery) y CD (deployment)**
sin mezclarlos, que es el error más común
- 4 Describir un pipeline etapa por etapa**
desde el commit hasta el monitoreo en producción
- 5 Ubicar Git, Docker, Kubernetes y Terraform**
cada uno en su lugar del ciclo de vida

Si al final de la clase alguno de estos cinco puntos te queda flojo, avisame antes de irte.

01

BLOQUE 1 · 20 MINUTOS

Por qué existe DevOps

Antes de las herramientas hay un problema organizacional. Si no lo entendés, DevOps queda en instalar Jenkins.

El muro de la confusión

Dos equipos con incentivos opuestos, separados por un ticket

DESARROLLO

Lo evalúan por features entregadas

- Quiere cambiar rápido y seguido
- El entorno de producción es una caja negra
- "En mi máquina funciona"

OPERACIONES

Lo evalúan por uptime y estabilidad

- Todo cambio es un riesgo de caída
- Recibe código que nunca vio antes
- "Entonces no lo despluguemos"

El resultado

Nadie es dueño del software de punta a punta. Dev tira el release por encima del muro, Ops lo recibe a ciegas, y cuando algo falla a las 3 AM la conversación es sobre de quién es la culpa, no sobre cómo arreglarlo.

Qué costaba mantener ese muro

El modelo anterior: releases grandes, poco frecuentes y aterradores

**1 vez
por trimestre**

Frecuencia de deploy típica pre-DevOps

Semanas

Desde que el código está listo hasta que llega a producción

Fin de semana

Ventana de mantenimiento habitual para un release

La trampa del release grande

- Cuanto más grande el cambio, más probable que rompa algo
- Cuanto más rompe, más miedo da desplegar → menos deploys
- Menos deploys → cambios todavía más grandes. El círculo se cierra.

Entonces, ¿qué es DevOps?

Una cultura y un conjunto de prácticas que integran desarrollo y operaciones para entregar software de forma continua, confiable y con responsabilidad compartida.

LO QUE SÍ ES

- Una forma de organizar equipos y responsabilidades
- Automatizar todo lo repetible y propenso a error
- Medir para decidir, no para opinar
- El equipo que construye, también opera

LO QUE NO ES

- Un puesto de trabajo llamado "el DevOps"
- Un equipo nuevo puesto entre Dev y Ops
- Comprar Jenkins y dar por terminado el tema
- Sinónimo de "usar la nube"

El ciclo de vida DevOps

Un bucle sin fin: lo que aprendés operando vuelve a alimentar lo que planificás

DEV · construir

- 1 Plan**
Qué construimos
- 2 Code**
Git, ramas, PRs
- 3 Build**
Compilar y empaquetar
- 4 Test**
Automatizado

OPS · entregar y sostener

- 5 Release**
Aprobación y versionado
- 6 Deploy**
A producción
- 7 Operate**
Escalar, mantener
- 8 Monitor**
Métricas y logs

Del paso 8 se vuelve al 1: el monitoreo de producción es la mejor fuente de requerimientos que vas a tener.

El mismo ciclo, con las herramientas encima

La cultura DevOps se implementa por fases, y cada fase tiene su caja de herramientas



Fijate en el cruce del centro: ahí está INTEGRAR. Es el punto exacto donde Dev entrega a Ops, y es lo que automatiza el pipeline.

Fase por fase: qué pasa y con qué

Leé el bucle de izquierda a derecha; el centro es donde se cierra

PLAN

Definir qué se construye y priorizar

Jira · Trello · Azure Boards

CONSTRUIR

Compilar y empaquetar el artefacto

Maven · Gradle · npm

INTEGRAR

El nudo: unir todo y disparar el pipeline

Jenkins · GitHub Actions · GitLab CI

FUNCIONAR

Operar, escalar y mantener en producción

Kubernetes · Terraform

CÓDIGO

Escribir y versionar el código fuente

Git · GitHub · GitLab

TESTEAR

Verificar automáticamente que funciona

Selenium · JUnit · Jest

DESPLEGAR

Llevar la versión a un ambiente real

Ansible · Docker · Helm

MONITOREAR

Observar y alimentar de vuelta la fase Plan

Nagios · Prometheus · Grafana

Las fases son estables desde hace quince años. Las herramientas se reemplazan cada tres o cuatro. Aprendé las fases primero.

02

BLOQUE 2 · 30 MINUTOS

La cultura CALMS

Cinco pilares que funcionan como checklist de madurez. Ninguno funciona solo.

CALMS: los cinco pilares

Un acrónimo que se volvió el estándar de facto para evaluar madurez DevOps



Cultura

Culture

Responsabilidad
compartida y confianza



Automatización

Automation

Si es repetible, se
automatiza



Lean

Lean

Eliminar desperdicio, lotes
chicos



Medición

Measurement

Datos sobre opiniones



Compartir

Sharing

Conocimiento sin dueños

Regla práctica: si automatizás sin cambiar la cultura, obtenés los mismos silos de siempre, ahora con scripts.

Cultura y Automatización

Los dos pilares que más resistencia generan al implementarse



Cultura

- Responsabilidad compartida sobre el resultado, no sobre la etapa
- "You build it, you run it": el que escribe el código atiende el incidente
- Postmortems sin culpables: se busca la causa del sistema, no la persona
- Fallar está permitido; fallar dos veces igual, no



Automatización

- Todo lo que se hace más de dos veces igual, se automatiza
- Build, tests, despliegue, provisión de infra, rollback
- Objetivo real: no ahorrar tiempo, sino eliminar la variabilidad humana
- Un proceso automatizado es además documentación ejecutable

Lean, Medición y Compartir

Los tres pilares que sostienen la mejora en el tiempo



Lean

Eliminar desperdicio y trabajar en lotes chicos

Trabajo en curso limitado · entregas pequeñas y frecuentes · nada de features 'por las dudas' · el valor se mide cuando llega al usuario, no cuando se mergea



Measurement

Si no se mide, no se mejora

Métricas técnicas (DORA), de negocio y de experiencia del equipo · las métricas se usan para aprender, nunca para rankear personas



Sharing

El conocimiento no tiene dueños

Documentación viva · runbooks · postmortems públicos · pair programming · demos internas · si una sola persona sabe hacer algo, eso es un riesgo operativo

Los cinco pilares se refuerzan entre sí: sin medición no sabés si lo lean funcionó; sin sharing, la mejora muere con quien la descubrió.

Cómo se mide: las 4 métricas DORA

El estándar de la industria para saber si un equipo DevOps realmente funciona

MÉTRICA	QUÉ MIDE	EQUIPO DE ALTO DESEMPEÑO	EQUIPO DE BAJO DESEMPEÑO
Frecuencia de despliegue	Cada cuánto llega código a producción	Varias veces al día	Cada 1 a 6 meses
Lead time for changes	Del commit hasta producción	Menos de 1 hora	Más de 1 mes
Change failure rate	% de deploys que causan un fallo	0 % a 15 %	46 % a 60 %
Time to restore service	Cuánto tardás en recuperarte	Menos de 1 hora	Más de 1 semana

Velocidad y estabilidad no se contraponen: los equipos que despliegan más seguido también son los que menos rompen.

ACTIVIDAD 1 · 10 MINUTOS

Diagnóstico CALMS de tu equipo

Puntuá del 1 al 5 cada pilar en tu trabajo actual (o en el último equipo en el que estuviste). Después compartimos 3 o 4 casos por micrófono.

C ¿El equipo que escribe el código se entera cuando falla en producción?

A ¿Cuántos pasos manuales hay entre un merge y el usuario final?

L ¿Cuánto tiempo pasa entre que empiezan una feature y que alguien la usa?

M ¿Podrían decir hoy cuántos deploys hicieron el mes pasado?

S Si mañana falta la persona clave, ¿alguien más puede desplegar?

El pilar con menor puntaje es por donde conviene empezar. Casi siempre es C o S, no A.

Break

10 minutos

Volvemos con CI/CD

03

BLOQUE 3 · 30 MINUTOS

CI/CD en profundidad

Tres siglas que se usan como una sola y significan cosas distintas. Hoy las separamos.

Integración Continua (CI)

Cada cambio se integra a la rama principal varias veces al día, y cada integración se verifica automáticamente

Qué pasa en cada push

- 1 Se dispara el pipeline automáticamente
- 2 Compila / construye el artefacto
- 3 Corre los tests unitarios y de integración
- 4 Analiza calidad y seguridad del código (SAST)
- 5 Reporta el resultado al autor en minutos

LAS TRES REGLAS NO NEGOCIABLES

Integrar al menos una vez al día

Ramas de vida corta. Una rama de tres semanas ya no es integración continua.

El build tiene que ser rápido

Menos de 10 minutos. Si tarda más, la gente deja de esperarlo.

Si el build se rompe, se arregla ya

Nadie mergea sobre main roto. Es la prioridad número uno del equipo.

Continuous Delivery ≠ Continuous Deployment

Misma sigla, una diferencia de una sola decisión: quién aprieta el botón

CONTINUOUS DELIVERY

Siempre listo para desplegar

- El artefacto pasa todas las pruebas y queda validado
- El paso a producción lo dispara una persona
- Sirve cuando hay regulación, ventanas comerciales o aprobaciones

Botón manual al final

CONTINUOUS DEPLOYMENT

Sale a producción solo

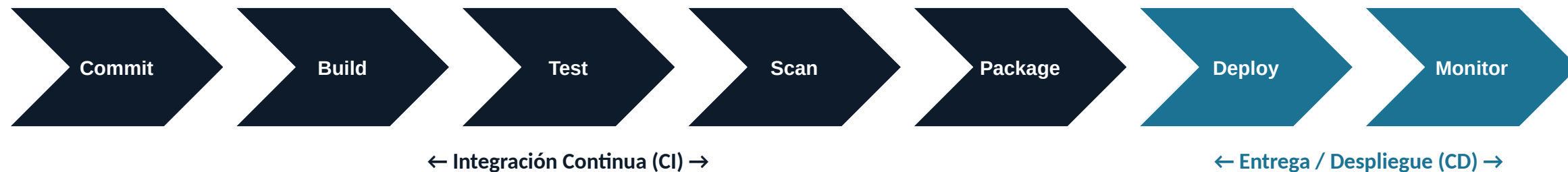
- Si el pipeline pasa en verde, va a producción sin intervención
- Exige una suite de tests en la que confíes de verdad
- Necesita feature flags y rollback automático como red de seguridad

Sin intervención humana

Todo Continuous Deployment es Continuous Delivery, pero no al revés. La mayoría de las empresas se queda —correctamente— en Delivery.

Anatomía de un pipeline

Del commit al monitoreo, sin tocar nada a mano



Commit

Push a Git. El evento dispara todo. Nada arranca a mano.

Build

Se compila y se genera el artefacto una única vez, reutilizado en todas las etapas.

Test

Unitarios, integración y end-to-end. Falla acá = no avanza.

Scan

SAST sobre el código y análisis de dependencias vulnerables.

Package

Se construye la imagen Docker y se publica en el registry con un tag versionado.

Deploy

Kubernetes recibe los manifiestos. Rolling update, sin downtime.

Monitor

Métricas, logs y alertas. Si algo se degrada, rollback.

Antipatrones que vas a encontrar en el mundo real

Todos parecen razonables desde adentro. Aprender a nombrarlos es medio camino.



"Después lo integramos"

Ramas que viven semanas. El merge final se convierte en un proyecto en sí mismo.



Tests que se saltean

Alguien agrega `--skip-tests` "solo por hoy" y queda para siempre. El pipeline verde deja de significar algo.



Deploy manual documentado en un wiki

Doce pasos, capturas de pantalla desactualizadas y una persona que sabe hacerlo.



Staging no se parece a producción

Todo pasa en staging y todo falla en prod. Distinta versión, distinta config, distinto todo.



Monitorear solo si el servidor está vivo

El ping responde, pero los usuarios no pueden comprar. Nadie se entera hasta que llaman.



El pipeline lo mantiene una sola persona

Cuando se va de vacaciones, nadie despliega. Clásico problema de Sharing.

04

BLOQUE 4 · 30 MINUTOS

El toolchain

Cuatro herramientas, cuatro problemas distintos. Panorama hoy, profundidad en los módulos 2 a 5.

Las cuatro piezas del toolchain

Cada una resuelve una pregunta distinta del ciclo de vida

Git

Módulo 2

Control de versiones distribuido

¿Cómo trabajamos varios sobre el mismo código sin pisarnos?

Repositorio · commit · branch · merge · pull request

Docker

Módulo 3

Contenerización

¿Cómo hago que corra igual en todos lados?

Imagen · contenedor · Dockerfile · registry

Kubernetes

Módulo 4

Orquestación

¿Cómo corro y escalo cientos de contenedores?

Pod · Deployment · Service · Ingress · HPA

Terraform

Módulo 5

Infraestructura como código

¿Cómo creo la infraestructura sin clickear en la consola?

Provider · resource · HCL · state

Cómo se encadenan en un flujo real

Un cambio de código, de tu editor a producción

1 Escribís código y hacés push

```
git push origin feature/login
```

2 El pipeline construye la imagen

```
docker build -t app:v1.2 .
```

3 Se publica en el registry

```
docker push registry/app:v1.2
```

4 Terraform garantiza que la infra exista

```
terraform apply
```

5 Kubernetes despliega la nueva versión

```
kubectl apply -f deployment.yaml
```

6 Prometheus y Grafana observan el resultado

```
métricas · logs · alertas
```

Ninguno de estos pasos lo ejecuta una persona a mano. Todos los dispara el pipeline a partir del push inicial.

DEMO EN VIVO · 15 MINUTOS

De cero a contenedor corriendo

Seguime en paralelo si tenés Git y Docker instalados. Si no, mirá ahora y replicalo con la guía después.

git — control de versiones

```
# 1. Versionar
git clone <repo>
git checkout -b feature/hola
echo "<h1>Hola DevOps</h1>" > index.html
git add .
git commit -m "feat: landing"
git push origin feature/hola
```

docker — contenedores

```
# 2. Contenerizar
cat Dockerfile
  FROM nginx:alpine
  COPY index.html \
    /usr/share/nginx/html/

docker build -t hola-devops:v1 .
docker run -d -p 8080:80 hola-devops:v1
docker ps
```

Cerrá abriendo localhost:8080 en el navegador. Ese es el momento en el que hace clic el concepto.

Actividad 2: chequeo rápido

Verdadero o falso. Respondé por el chat, después las revisamos juntos.

1

DevOps es un puesto de trabajo dentro del equipo de infraestructura.

V / F

2

La "M" de CALMS se refiere a medición de métricas técnicas y de negocio.

V / F

3

En Continuous Deployment una persona aprueba manualmente el pase a producción.

V / F

4

Los equipos que despliegan más seguido tienden a tener menos fallas.

V / F

5

Un contenedor incluye un sistema operativo completo, igual que una máquina virtual.

V / F

6

El mismo artefacto que pasó los tests es el que debe llegar a producción.

V / F

Las respuestas están en el guión docente. Cada una es una excusa para repasar el concepto, no para evaluar.

Para la próxima clase

Dos consignas cortas, no más de 45 minutos en total

1 Preparar el entorno

- Instalar Git y verificar con `git --version`
- Instalar Docker Desktop y verificar con `docker --version`
- Crear cuenta en GitHub si no tenés
- Correr `docker run hello-world` y guardar la captura

2 Diagnóstico escrito

- Media carilla sobre un equipo real (el tuyo o uno conocido)
- Puntuar los 5 pilares CALMS del 1 al 5 y justificar el más bajo
- Proponer una acción concreta para mejorarlo en 30 días

Próxima clase — Módulo 2: Dominando CI/CD y automatización con Git

Ramas, merges, resolución de conflictos y tu primer workflow de GitHub Actions.

Lo que nos llevamos

- 1 DevOps es cultura antes que herramientas: nació para derribar el muro entre Dev y Ops
- 2 CALMS da el checklist: cultura, automatización, lean, medición y compartir
- 3 DORA da los números: frecuencia, lead time, tasa de fallas y tiempo de recuperación
- 4 CI es integrar seguido y verificar automáticamente; CD tiene dos sabores, delivery y deployment
- 5 Git, Docker, Kubernetes y Terraform resuelven cuatro problemas distintos del mismo ciclo

¿Preguntas?